

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório de Especificação da Fase C
Projeto Integrador em Telecomunicações e Informática

Protótipo De Um Sistema De Entretenimento (e utilitário) Num Aeroporto

Grupo 3 constituído por:

Luís Pedro Cerqueira Baptista, PG61434

Bernardo Gualdino Salgado, PG59972

Nuno Duarte Mirra, PG57594

Pedro João Daniel, PG58741

2025/2026

Índice

1. Introdução	4
2. Arquitetura do Sistema e Protocolo de Aplicação	4
2.1. Visão Geral da Arquitetura	4
2.2. Especificação da Aplicação	5
2.3. Gestão de Ficheiros	9
2.4. Estratégia de Multiplexagem e Concorrência	9
3. Desenvolvimento da Aplicação de Entretenimento	11
3.1. Identificação das Aplicações e Serviços	11
3.2. O Protocolo de Aplicação como Multiplexador	11
3.3. Especificação da Interface Web (Frontend)	12
3.4. Lógica de Backend, Multithreading e APIs	12
3.5. Transferência de múltiplos ficheiros	13
4. Implementação, Integração e Desempenho	13
4.1. Análise de Desempenho e Otimização	14
4.2. Fiabilidade e Integridade na Aplicação	14
5. Competências necessárias e Planeamento	15
5.1. Competências Técnicas	15
5.2. Planeamento de Tarefas	15
6. Conclusão	16
7. Referências	17

Índice de Figuras

Figura 1 - Diagrama de Blocos	5
Figura 2 – Formato da mensagem de pedido ao servidor por parte do cliente.....	6
Figura 3 - Formato da mensagem da resposta do servidor ao pedido do cliente.....	6
Figura 4 - Formato da mensagem para controlo de ficheiros e erros.	7
Figura 5 - Formato da mensagem para o envio de ficheiros por parte do servidor.....	7
Figura 6 - Diagrama de Sequência	10
Figura 7 - Diagrama de Gant	16

1. Introdução

Nesta última etapa (Fase C) do trabalho, o objetivo principal consiste na implementação da Camada de Aplicação (Nível 7 do modelo OSI) e na respetiva integração com as camadas de Ligação de Dados e Física desenvolvidas anteriormente. O sistema final é um protótipo de comunicações óticas sem fios (Li-Fi) desenhado para funcionar como um sistema de consulta de informações num aeroporto.

Além de garantir o acesso em tempo real a dados de voos e permitir a transferência de documentos digitais (como mapas ou revistas), nesta fase o sistema foi adaptado para um modelo Cliente-Servidor em modo Full-Duplex. A interação será feita por uma Interface Gráfica (GUI), desenvolvida com tecnologias Web e integrada com o backend através de WebSockets.

Este relatório detalha a especificação do protocolo de aplicação desenvolvido pelo grupo, capaz de gerir diferentes tipos de dados em simultâneo e sem bloqueios, mantendo a independência entre as camadas da rede.

2. Arquitetura do Sistema e Protocolo de Aplicação

2.1. Visão Geral da Arquitetura

Na arquitetura da Camada de Aplicação (Nível 7) o software desenvolvido interage com o sistema de comunicação através da API definida na Fase B (`enviar_bloco_dados` e `processar_trama_recebida`), e a camada 2 atua como um canal de transporte abstraído, desconhecendo se o payload transportado corresponde a uma consulta de voo ou a um documento digital.

Deste modo, e da mesma forma, a Camada 7 abstrai-se totalmente dos mecanismos de retransmissão (ARQ) e controlo de erros. O diagrama da **Erro! A origem da referência não foi encontrada.** mostra como é que a Camada 7 faz a multiplexagem e gere a concorrência antes de entregar os pacotes à API da Camada 2.

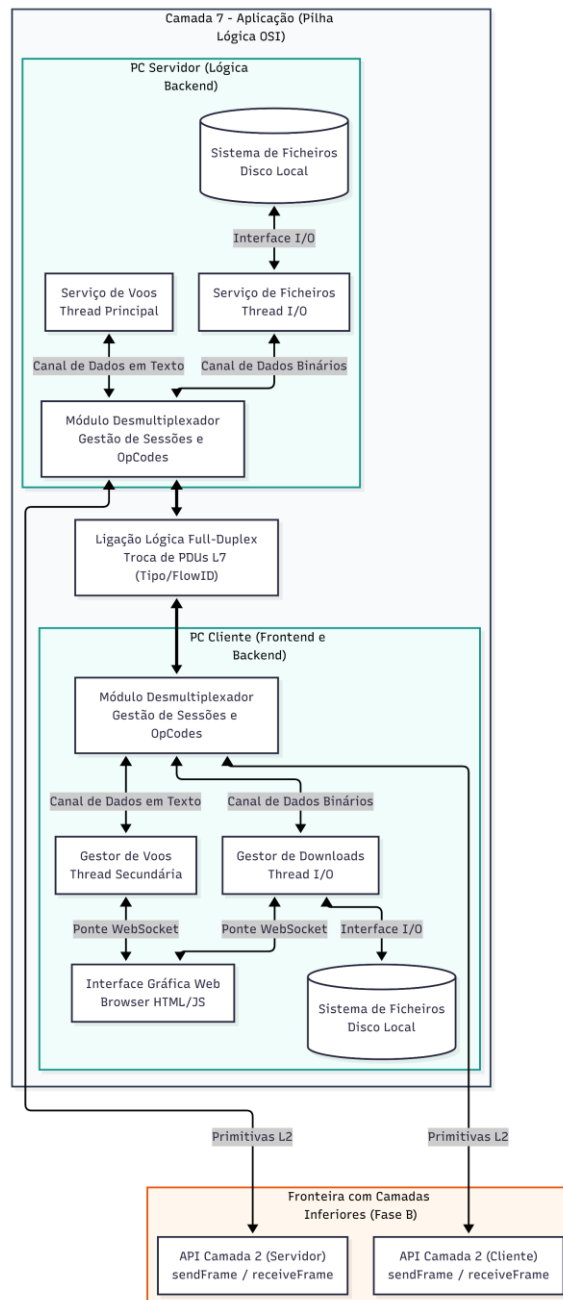


Figura 1 - Diagrama de Blocos

2.2. Especificação da Aplicação

Na especificação da Camada de Aplicação (Camada 7), a estrutura das Unidades de Dados de Protocolo (PDUs) é variável, adaptando-se ao tipo de operação e à direção da comunicação. Esta abordagem permite otimizar a organização das mensagens, garantindo que as tramas transportam apenas os campos necessários para cada contexto, o que reduz o peso dos cabeçalhos.

Relativamente ao dimensionamento das mensagens, foi necessário considerar uma limitação prática imposta pelas camadas inferiores. Embora a Camada de Ligação de Dados (Camada 2) suporte um payload máximo de 256 bytes, a aplicação do algoritmo de correção de erros FEC (Código de Hamming 7,4) implica que cada nibble de 4 bits ocupe um byte inteiro durante a transmissão. Na prática, esta codificação duplica o volume ocupado pelos dados originais. Consequentemente, para assegurar que a trama final não ultrapassa o limite físico da Camada 2, o tamanho total das PDUs da Camada 7 ficou limitado a um máximo de 128 bytes (cabeçalho da aplicação + dados úteis).

Para facilitar o tratamento dos dados, todas as PDUs começam obrigatoriamente com um cabeçalho de 2 bytes: Tipo (identifica o serviço, ex: Ficheiro ou Info) e Subtipo (identifica a ação, ex: Pedido, Resposta ou Dados). Com este modelo, definiram-se quatro formatos de mensagem:

A. Formato de Pedido (Cliente para o Servidor)

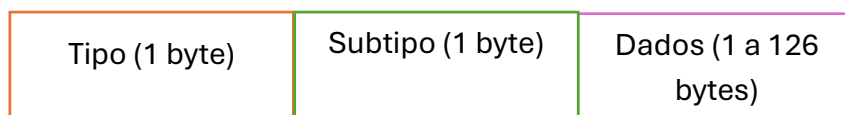


Figura 2 – Formato da mensagem de pedido ao servidor por parte do cliente.

Utilizado pelo Cliente para solicitar informações ou o descarregamento de documentos. Como a mensagem é curta e não necessita de ser dividida em fragmentos, o cabeçalho é simples, o que deixa mais espaço livre para os dados do pedido (como o nome do ficheiro ou o código do voo).

- Tipo (1 byte): Define o serviço.
- Subtipo (1 byte): Especifica a ação (Pedido ou Consulta).
- Dados (1 a 126 bytes): Parâmetro de busca em texto UTF-8 (ex: "mapa.pdf" ou "TP123").

B. Formato de Resposta (Servidor para o Cliente)

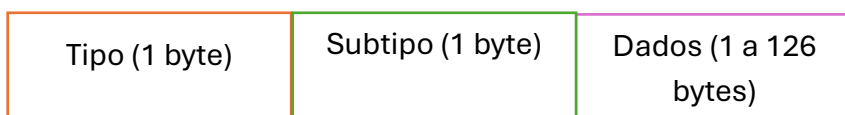


Figura 3 - Formato da mensagem da resposta do servidor ao pedido do cliente.

Utilizado pelo Servidor para responder aos pedidos de informação por parte do cliente. Como as mensagens são apenas texto simples e não necessitam de ser fragmentadas, o cabeçalho é mais pequeno e não inclui os campos que os ficheiros usam (como o ID de fluxo ou o ID de fragmento).

- Tipo (1 byte): Serviço de Informação.

- Subtipo (1 byte): Resposta.
- Dados (1 a 126 bytes): Mensagem de resposta codificada em UTF-8 (ex: "Embarque - Porta 4").

C. Formato de Controlo de Ficheiros e Erros (Bidirecional)

Tipo (1 byte)	Subtipo (1 byte)	ID de Fluxo (1 byte)	Dados (1 a 125 bytes)
---------------	------------------	----------------------	-----------------------

Figura 4 - Formato da mensagem para controlo de ficheiros e erros.

Utilizado para a gestão da transferência (atributos do ficheiro, finalização ou cancelamento) e para indicar falhas. O cabeçalho contém um ID de Fluxo para identificar o processo em causa:

- Tipo (1 byte): Ficheiro ou Erro.
- Subtipo (1 byte): Metadados, Fim de Ficheiro, Abortar ou Erro (ex: Not Found).
- ID de Fluxo / FlowID (1 byte): Identifica a qual ficheiro a mensagem de controlo pertence (ex: FlowID: 1 para o Mapa), permitindo sinalizar exatamente a transferência em causa.
- Dados (1 a 125 bytes): Informação de controlo (ex: "Tam: 500KB", "Total=N" ou "Erro 404").

D. Formato de Fragmento de Ficheiro (Servidor para o Cliente)

Tipo (1 byte)	Subtipo (1 byte)	ID de Fluxo (1 byte)	ID de Fragmento (4 bytes)	Dados (1 a 121 bytes)
---------------	------------------	----------------------	---------------------------	-----------------------

Figura 5 - Formato da mensagem para o envio de ficheiros por parte do servidor.

Utilizado para o envio dos fragmentos de dados. O cabeçalho ocupa 7 bytes para permitir a gestão de várias transferências em simultâneo e para suportar ficheiros de grande dimensão:

- Tipo (1 byte): Ficheiro.
- Subtipo (1 byte): Dados.
- ID de Fluxo / FlowID (1 byte): Permite a transferência múltipla de ficheiros em simultâneo através da multiplexagem do canal.
- ID de Fragmento / FragID (4 bytes): Número de sequência. A utilização de 4 bytes permite que o sistema suporte ficheiros com mais de 1 GB, garantindo a ordem correta na escrita e a retoma em caso de falha.

- **Dados Binários (1 a 121 bytes):** O conteúdo real do ficheiro.

Para otimizar a lógica de desmultiplexagem, a identificação das operações suportadas pelo sistema encontra-se estruturada nos seguintes módulos:

- **Módulo 0x10 (Informação de Voos):** Este grupo trata da troca de mensagens de texto simples. Ao contrário dos ficheiros, estes dados são geridos num buffer temporário para serem enviados para a interface gráfica através de WebSockets.
 - **Tipo 0x10 | Subtipo 0x01 (Pedido de Informação de Voo):** Enviado pelo cliente para consultar o estado de um voo através de uma string (ex: "TP123").
 - **Tipo 0x10 | Subtipo 0x02 (Resposta de Informação de Voo):** Enviado pelo servidor para indicar o estado atualizado do voo solicitado.
- **Módulo 0x20 (Transferência de Ficheiros):** Agrupa as operações que exigem interação assíncrona com o sistema de ficheiros local (leitura e escrita no disco).
 - **Tipo 0x20 | Subtipo 0x01 (Pedido de Documento):** Utilizado pelo cliente para solicitar o descarregamento de um ficheiro específico pelo nome.
 - **Tipo 0x20 | Subtipo 0x02 (Atributos do Ficheiro):** Emitido pelo servidor antes da transferência dos fragmentos, transportando o tamanho total e o nome do documento para preparar o recetor.
 - **Tipo 0x20 | Subtipo 0x03 (Envio de Fragmentos):** Utilizado para transmitir os fragmentos do ficheiro. Os campos FlowID e FragID permitem ordenar os dados e escrever no disco à medida que estes chegam.
 - **Tipo 0x20 | Subtipo 0x04 (Fim de Ficheiro):** Indica que a transferência terminou. Contém o número total de fragmentos enviados para confirmar que não houve falhas antes de guardar o ficheiro no disco.
 - **Tipo 0x20 | Subtipo 0x05 (Abortar Transferência):** Permite cancelar a operação a meio, interrompendo a escrita e eliminando os dados temporários para libertar espaço.
- **Módulo 0x30 (Erros da Aplicação):** Este módulo serve para separar os erros de funcionamento do fluxo normal de dados do protocolo.
 - **Tipo 0x30 | Subtipo 0x01 (Erro de Aplicação / Not Found):** Utilizado para indicar falhas, como ficheiros inexistentes no servidor ou erros de escrita. O campo de dados contém uma descrição do erro para informar o utilizador sobre o motivo da interrupção.

2.3. Gestão de Ficheiros

A transferência de documentos digitais foi planeada para garantir que o ficheiro é reconstruído no disco sem erros e com as suas características originais. Para evitar a utilização de valores fixos no código (*hard coded*), o processo foi dividido em três fases:

- **Envio de Atributos do Ficheiro:** O processo começa com o envio das informações iniciais do ficheiro (**Tipo 0x20 | Subtipo 0x02**). Esta trama transporta o nome e o tamanho total do documento, permitindo que o sistema crie o ficheiro vazio no disco local e se prepare para receber os dados.
- **Transferência de Fragmentos:** De seguida, são enviadas as tramas com o conteúdo real (**Tipo 0x20 | Subtipo 0x03**). O sistema utiliza o **FragID (4 bytes)** de cada pacote para garantir a ordenação correta, escrevendo os dados no disco à medida que estes chegam. Esta escrita sequencial é mais eficiente e permite gerir ficheiros de grande dimensão sem ocupar a memória desnecessariamente.
- **Finalização e Validação do Ficheiro:** No final, é enviada uma trama que sinaliza a conclusão da transferência (**Tipo 0x20 | Subtipo 0x04**). Esta mensagem contém o número total de fragmentos que deveriam ter sido entregues. O sistema compara este valor com a contagem dos blocos que foram efetivamente escritos no disco, se os números coincidirem, a transferência do ficheiro é dada como finalizada e o ficheiro é guardado com sucesso.

2.4. Estratégia de Multiplexagem e Concorrência

Para permitir a multiplexagem e a concorrência (*multithreading*), o sistema foi estruturado de forma que a transferência de ficheiros não bloqueie a interface de consulta de voos. Esta arquitetura assenta numa separação clara entre o cliente e o servidor.

- **Módulo de Interface Cliente (GUI):** Responsável pela gestão das interações e pela atualização dos dados apresentados ao utilizador.
- **Módulo de Comunicação:** Encarregue de manter um ciclo contínuo de leitura da porta série (utilizando a API da Camada 2) e de gerir a transmissão de pacotes.

A separação das mensagens é feita através da verificação dos campos Tipo e Subtipo. Se o módulo de comunicação receber um pacote de um ficheiro, os dados seguem para a escrita no disco. Se os dados forem relativos a um voo, são enviados para a interface gráfica.

Esta organização permite que ambos os serviços funcionem ao mesmo tempo no mesmo canal físico a 9600 bps. Como se observa no Diagrama de Sequência (Figura 5), a interface permanece operacional mesmo durante a recepção de documentos. Além disso, a lógica de processamento (como o cálculo do número total de fragmentos) é feita no servidor, e os atributos do ficheiro (nome e tamanho) são enviados logo no início para evitar o uso de valores fixos no código.

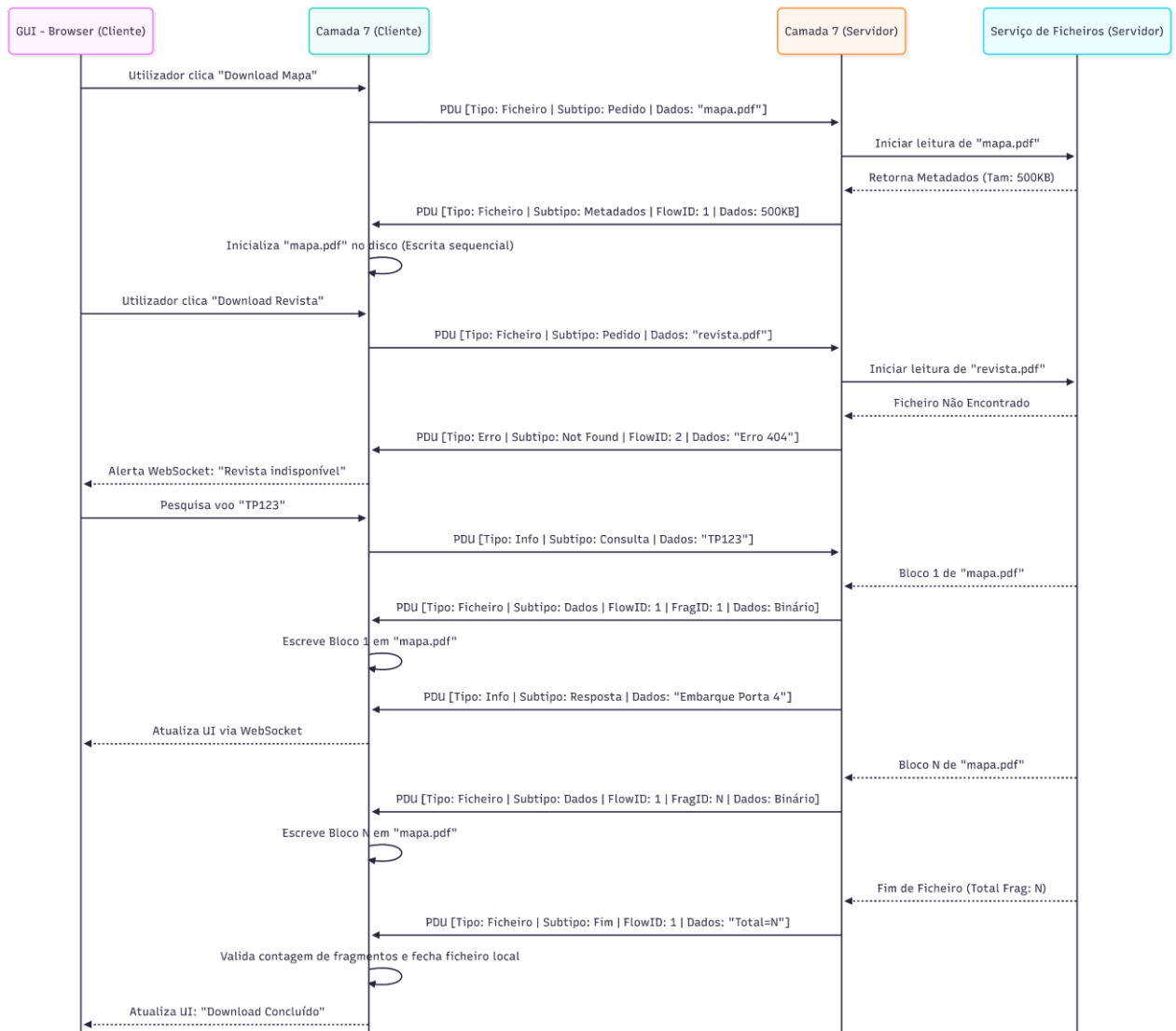


Figura 6 - Diagrama de Sequência

3. Desenvolvimento da Aplicação de Entretenimento

3.1. Identificação das Aplicações e Serviços

O sistema organiza-se em dois serviços principais que funcionam em simultâneo, com características e requisitos distintos:

- **Serviço A (Gestor de Informação de Voos):** É responsável pela troca de informações de texto simples. O fluxo baseia-se num pedido de consulta por parte do cliente (ex: o voo "TP123"), ao qual o servidor responde com o estado atualizado (ex: "Porta 4 - Embarque").
- **Serviço B (Gestor de Ficheiros Digitais):** Trata do envio de documentos com maior volume, como mapas ou imagens em formato .pdf. Ao contrário do serviço anterior, este exige a realização de operações de leitura e escrita no disco. Por este motivo, o serviço corre em segundo plano para evitar que a interface ou os restantes processos do sistema fiquem bloqueados durante a transferência.

3.2. O Protocolo de Aplicação como Multiplexador

Para permitir que os diferentes serviços funcionem ao mesmo tempo num canal de 9600 bps, implementou-se um sistema de multiplexagem. Tal como em redes TCP/IP a separação é feita por portas (como a porta 80 para HTTP), no protocolo desenvolvido esta função é assegurada pelos campos Tipo e Subtipo do cabeçalho. O desmultiplexador da Camada 7 encaminha as mensagens recebidas conforme a sua finalidade:

- Se a PDU tiver um Tipo do módulo **0x10**, os dados seguem para o processamento do Serviço A (Informação de Voos).
- Se tiver um Tipo do módulo **0x20**, o pacote é enviado para o Serviço B, que trata da escrita dos dados no disco.

Esta organização garante que cada serviço processe apenas os dados que lhe dizem respeito, permitindo que as consultas e as transferências de ficheiros ocorram em paralelo, sem conflitos ou bloqueios na comunicação

3.3. Especificação da Interface Web (Frontend)

Para evitar as limitações visuais de uma consola de terminal e conferir um aspeto realista de painel de aeroporto, a interface gráfica do cliente foi concebida como uma Single Page Application (SPA), executada localmente no browser através das seguintes tecnologias:

- HTML e CSS: Definem a estrutura visual e o estilo do painel, separando a zona de consulta de voos da área de descarregamento de documentos digitais.
- JavaScript e WebSockets: O motor do lado do cliente (Frontend) não realiza pedidos HTTP tradicionais. Em vez disso, estabelece uma ligação WebSocket bidirecional permanente com o Backend. Esta tecnologia permite a manipulação do DOM (Document Object Model) em tempo real, atualizando o estado de um voo ou a barra de progresso de um ficheiro instantaneamente, sem necessidade de recarregar a página.

3.4. Lógica de Backend, Multithreading e APIs

A lógica de rede (Camadas 2 e 7) foi desenvolvida em Python. De modo a suportar a concorrência, a arquitetura baseia-se em multithreading, o que permite separar as diferentes tarefas:

- Thread de Interface (Servidor WebSocket): Destina-se à escuta de pedidos provenientes do navegador (browser) e ao envio de atualizações de estado em tempo real.
- Thread de Ficheiros (Gestão do Serviço B): Fica responsável pelas operações de escrita no disco. Esta componente trata da manipulação de ficheiros em modo binário e anexa os blocos de dados recebidos, concluindo o processo assim que o sinal de fim de ficheiro é detetado.
- Thread de Rede (Camada 2 / UART): Executa um ciclo contínuo dedicado à comunicação com o hardware e à Camada de Ligação. Esta componente utiliza a biblioteca pyserial para a leitura e escrita de bytes na porta série, realizando o cálculo do CRC-8 para deteção de erros e a gestão da máquina de estados do protocolo ARQ (Stop-and-Wait).

O código foi estruturado de forma a assegurar que as diferentes camadas funcionam de forma independente. A troca de informação é realizada exclusivamente através de APIs internas (por exemplo, a Camada 7 invoca a função `enviar_bloco_dados`

que é disponibilizada pela Camada 2). Desta forma, a Camada de Ligação de Dados processa as tramas sem interpretar o tipo de conteúdo transportado (seja texto de voos ou fragmentos de um ficheiro), enquanto a Camada de Aplicação ignora os detalhes das retransmissões e temporizadores geridos nas camadas inferiores.

3.5. Transferência de múltiplos ficheiros

Para permitir o envio de vários ficheiros em simultâneo, o protocolo utiliza o campo FlowID. Este identificador, inserido no cabeçalho de cada trama, permite distinguir a que documento pertence cada fragmento de dados recebido.

O envio é gerido através do Round-Robin. O servidor transmite um bloco do primeiro ficheiro, passa para um bloco do segundo e continua o ciclo até que todos os documentos sejam entregues. Esta estratégia garante que o canal é partilhado de forma equilibrada, evitando que a transferência de um ficheiro muito grande impeça a chegada de outros documentos ou de mensagens de texto

4. Implementação, Integração e Desempenho

A implementação final do sistema reúne os desenvolvimentos das três fases do projeto, assegurando que o hardware e o software funcionem num modelo Cliente-Servidor em Full-Duplex. A integração foi organizada de modo a assegurar a separação entre níveis, distribuindo as responsabilidades da seguinte forma:

1. **Integração Hardware-Software (Camada 1):** O hardware desenvolvido na Fase A funciona como a interface ótica. O sinal digital UART (0-3.3V) gerado no PC é convertido em sinais infravermelhos (LED) e captado pelo fotodetector. Nesta fase, a ESP32 atua como suporte físico, garantindo que o feixe ótico atinja o alcance mínimo de 1 metro exigido.
2. **Abstração e Independência (Camada 2):** Implementada em Python no PC, esta camada faz a gestão do canal lógico bidirecional. É responsável pela construção de tramas com CRC-8 e pela execução do protocolo ARQ (Stop-and-Wait). Seguindo o princípio da independência entre camadas, a Camada 2 transporta os PDUs da aplicação sem analisar o seu conteúdo, quer se trate de atributos de um ficheiro ou de informações de voo.
3. **Gestão de Fluxos e Multiplexagem (Camada 7):** A Camada de Aplicação utiliza a API da Camada 2 (`enviar_bloco_dados` e `processar_trama_recebida`) para gerir os serviços disponibilizados. Através da análise do OpCode, o sistema encaminha as mensagens: as consultas de voo são processadas em memória para atualização

da interface Web, enquanto a transferência de ficheiros é gerida por threads que realizam a escrita no disco local.

4.1. Análise de Desempenho e Otimização

O sistema opera a uma taxa de 9600 baud, o que, considerando o formato UART 8-N-1 (10 bits por byte), resulta numa taxa de transmissão de 960 bytes/segundo. Para otimizar o desempenho, foram adotadas as seguintes estratégias:

1. **Payload Dinâmico:** Em vez de pacotes de tamanho fixo, foi implementado um sistema de PDU com tamanho variável que se adapta até ao limite máximo de 128 bytes. O envio de blocos na sua capacidade máxima reduz o peso dos cabeçalhos face aos dados úteis e diminui o número de paragens do protocolo Stop-and-Wait à espera de confirmações (ACKs). Esta abordagem melhora significativamente a taxa de transferência real (throughput) do sistema.
2. **Eficiência na Multiplexagem (Round-Robin):** Para evitar que o envio de um ficheiro grande ocupe o canal sozinho e impeça a consulta de voos, o sistema alterna entre os serviços. O servidor envia um fragmento de dados e, logo de seguida, verifica se existe algum pedido de texto pendente para responder. Esta partilha do canal garante que a interface do utilizador nunca deixa de responder, mesmo quando estão a ser transferidos ficheiros de 1 GB.

4.2. Fiabilidade e Integridade na Aplicação

A nível da aplicação, a fiabilidade foca-se em garantir que os dados reconstruídos no disco correspondem exatamente ao ficheiro original, independentemente das interrupções que possam ocorrer no canal físico.

Para assegurar esta resiliência, o protocolo de aplicação utiliza os seguintes mecanismos:

- **Validação do número total de fragmentos recebidos:** No final de cada transferência, o sistema realiza uma validação comparando o número total de fragmentos enviados pelo servidor com os blocos que foram efetivamente escritos no disco. Esta confirmação final é essencial para garantir que o ficheiro está completo antes de ser dado como concluído.
- **Ordenação dos fragmentos:** Através da utilização do FragID (4 bytes), a aplicação consegue colocar cada pedaço de informação na sua posição exata. Se a ligação ótica cair e for retomada mais tarde, o sistema continua a escrita a partir do ponto onde parou, evitando que o ficheiro fique corrompido ou com dados trocados.
- **Independência e Recuperação:** Seguindo o princípio da independência entre camadas, a aplicação não precisa de gerir retransmissões. Ela limita-se a processar os dados que lhe são entregues pelas camadas inferiores. Caso ocorra uma interrupção prolongada ou um erro irreversível, o comando de Abortar permite limpar os dados temporários e libertar os recursos do sistema.

5. Competências necessárias e Planeamento

5.1. Competências Técnicas

A fase C requer competências técnicas além do processamento de sinal e arquitetura de redes, abrangendo agora engenharia de software de alto nível:

- Domínio de threads e gestão de operações de leitura e escrita (I/O) sem bloqueio para evitar que a aplicação deixe de funcionar.
- Gestão de Armazenamento de Dados
- Desenvolvimento de Interfaces Gráficas (GUI)

5.2. Planeamento de Tarefas

Tendo em conta que a demonstração da Fase C está agendada para 29 de maio, o cronograma da Fase C organiza-se no seguinte diagrama de Gantt representado na **Figura 7**.

T1 - Especificação e Design da Camada de Aplicação:

- Definir a estrutura da Unidade de Dados de Protocolo (PDU) da Camada 7 e os respetivos identificadores de operação (OpCodes).
- Projetar o mecanismo de extração e envio de metadados, assegurando que o nome e o tamanho dos ficheiros não ficam hard coded no código.
- Planear a lógica de multiplexagem para suportar a transferência simultânea de informações de voo e ficheiros.

T2 - Desenvolvimento do Protocolo e Gestão de Ficheiros:

- Programar a lógica de fragmentação de dados no emissor e a correspondente remontagem de blocos no recetor.
- Implementar a leitura binária e a escrita sequencial de ficheiros digitais (mapas, revistas) no disco local.
- Garantir o princípio de independência entre camadas, garantindo que a Camada 7 não tem conhecimento das tramas de controlo trocadas na Camada 2.

T3 - Implementação da Interface Gráfica (GUI) e Concorrência:

- Desenvolver a interface de utilizador do lado do recetor para a consulta do sistema de informações do aeroporto.
- Implementar multithreading ou concorrência assíncrona para assegurar que a aplicação não bloqueia durante a transferência de ficheiros.
- Integrar a receção de dados em tempo real com a atualização visual da interface.

T4 - Integração de Sistemas e Otimização:

- Integrar todo o hardware e software desenvolvido nas Fases A, B e C num único sistema de comunicação.
- Otimizar o código para reduzir o tempo de transferência ao mínimo teórico possível face aos 9600 bps da porta série.
- Afinar e validar a transferência fiável de dados em cenários de interrupção momentânea do feixe ótico.

T5 - Testes Finais, Documentação e Preparação:

- Executar testes com ficheiros de imagem e texto de diferentes tamanhos.
- Escrever o relatório final focado na implementação e otimização da solução.
- Preparar o código final e a apresentação oral para a demonstração final.

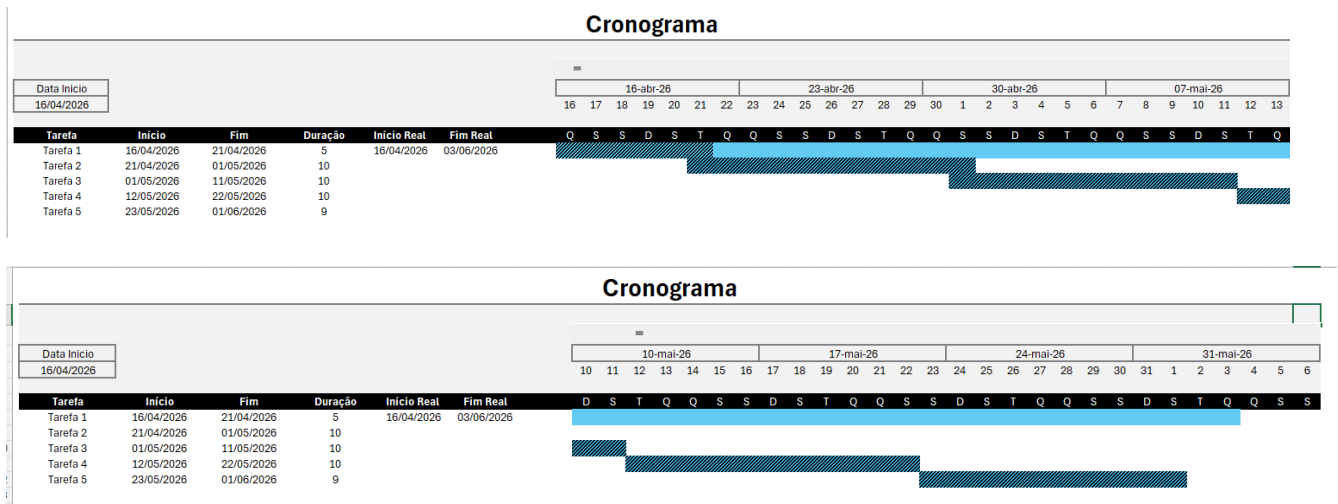


Figura 7 - Diagrama de Gant

6. Conclusão

A especificação da Fase C detalha a integração das várias componentes desenvolvidas ao longo do semestre, definindo um sistema que visa demonstrar a viabilidade de uma pilha protocolar OSI sobre um canal de comunicação ótico. A arquitetura definida para a camada de aplicação foca-se na criação de um serviço de aeroporto capaz de disponibilizar informações de voo em tempo real e de garantir a transferência fiável de ficheiros binários.

A configuração Cliente-Servidor em Full-Duplex, baseada na API da fase anterior, foi projetada para garantir a fiabilidade da comunicação entre o cliente e o servidor. A

escolha de uma interface Web com WebSockets tem como objetivo, verificar o funcionamento da multiplexagem e da concorrência (multithreading), de modo que a interface se mantenha operacional durante o descarregamento de documentos em segundo plano.

Em suma, a estratégia de integração apresentada assegura a independência entre as camadas da rede. O protocolo de aplicação está estruturado para funcionar sem depender das variações do meio físico ou dos mecanismos de retransmissão geridos pela Camada 2. A solução proposta visa garantir a recuperação automática perante falhas temporárias no canal ótico, assegurando a integridade dos ficheiros e o cumprimento dos objetivos definidos para a fase final do projeto.

7. Referências

- [1] I. Lau, S. Ekpo, M. Zafar, M. Ijaz, e A. Gibson, «Hybrid mmWave-Li-Fi 5G Architecture for Reconfigurable Variable Latency and Data Rate Communications», *IEEE Access*, vol. 11, pp. 42850–42861, 2023, doi: 10.1109/ACCESS.2023.3270777.
- [2] L. I. Albraheem, L. H. Alhudaithy, A. A. Aljaser, M. R. Aldhafian, e G. M. Bahliwah, «Toward Designing a Li-Fi-Based Hierarchical IoT Architecture», *IEEE Access*, vol. 6, pp. 40811–40825, 2018, doi: 10.1109/ACCESS.2018.2857627.